Character Pointer & Strings in C

A character pointer is a pointer(variable) that stores the address of a character variable.

```
char c = 'A';
char *p = &c;
```

Before moving to strings in C, it is very important to note that string is a sequence of characters terminated with a null character '\0'.

Note the following difference :

```
char ch[] = {'a', 'b', 'c'};
// it is an array of characters

char ch[] = {'a', 'b', 'c', '\0'};
// it is treated as string(sequence of characters terminated with null character, i.e. \0)
```

NOTE : How to print string?

```
char str1[] = "Hello";
char *str2 = "Hi";
char str3[10] = "Hola";

printf("%s\n%s\n%s\n", str1, str2, str3);
```

```
Output : Hello
         Hi
         Hola
```

Note that we are using "%s" as format specifier to print the string.

NOTE : How to take string input?

The same format specifier "%s" is used with scanf() to take the input to the string.
But the catch is, it won't include whitespaces.
E.g. char ch[] = "Hello world";
printf("%s", ch);
The output will be "Hello" only. All the characters after the first whitespace will not be taken to the input.

Solution : To avoid the above problem, we use "**%[^\0]s**" format specifier.
%[...] -> scan set (read characters that matches this set)
^ -> negation(reads characters not in the set)
"**%[^\0]s**" -> reads characters until a newline is encountered

We can also use **gets()**function to take string input. But **gets()**function has been deprecated. Better not to use it in your program.

NOTE : Different Ways to Initialise String in C

1. **String literal (Pointer initialisation)**

```
char *str = "Hello";
str = "Hi"; // re-initialisation is allowed
str[1] = 'a'; // modification not allowed
```

2. **Character Array with String Literal**

```
char str[] = "Hello";
str[1] = 'A'; // modification allowed
str = "Hi"; // re-initialisation not allowed
```

3. **Character Array with Explicit Characters + '\0'**

```
char str[] = {'a', 'b', 'c', '\0'};
-> Modifiable ( str[1] = 'a'; }
-> Cannot be re-initialised
```

4.) Character Array Without '\0' (null character termination)

```
char str[] = {'a', 'b', 'c'};
-> It is NOT a string
-> It is just an array of characters
```

5.) Fixed Size Array with String Literals

```
char str[10] = "Hello";
```

In the above case, the array will be initialised as :

```
{'H', 'e', 'l', 'l', 'o', '\0', '\0', '\0',
'\0', '\0'}
```

Rest of the elements are initialised with null character (\0).

Note that the string length must be lesser than the size of the array. Because '\0' (null character) is stored at the end of the string, leading to increase the size of the string by 1.

Q. What if we initialise the array with string of length greater than the size of the array?
A. It will cause undefined behaviour.

6.) Using malloc()

```
char *str = (char *)malloc(10*sizeof(char));
```

strcpy(str, “Hello”); // This will copy the string “Hello” to the array pointed by the pointer str.

str = “Hi”; // valid, but the memory allocated before will no longer be accessible(Memory Leakage)

Method	Example	Memory	Writable?	Reassignable?	Auto \0 ?
Pointer to literal	`char *s = "Hi";`	Read-only	✗	✓	✓
Array from literal	`char s[] = "Hi";`	Stack	✓	✗	✓
Array literal + \0	`{ 'H','i','\0' }`	Stack	✓	✗	✓
Array literal no \0	`{ 'H','i' }`	Stack	LIMITED	✗	✗
Fixed size + literal	`char s[10] = "Hi";`	Stack	✓	✗	✓
Uninitialized array	`char s[10];`	Stack	✓	✗	✗
strcpy into array	`strcpy(s,"Hi");`	Stack	✓	✗	✓
malloc + strcpy	`malloc()+strcpy()`	Heap	✓	✓	✓
malloc + literal	`s=malloc(); s="Hi";`	Heap+RO	✗	✓	✓

NOTE : Character Pointer vs String Literal

```
char *str = "hello";

String literal "hello\0" in read-only memory
      h  e  l  l  o  \0
      ↓
   +--------+
   |  str   | → address of 'h'
   +--------+
```

Note that the String Literal is stored in the Read-only memory, so the value of the literal cannot be modified if we have initialised the string using character pointer.

What does the following represent?
str -> address of the first element ‘h’
***str** -> value of the first element i.e. ‘h’
str + 1 -> address of next element of ‘h’, i.e. ‘e’
***(str + 1)** -> value of next element to ‘h’, i.e. ‘e’
str[2] -> 3rd character of the string, i.e. ‘l’

NOTE : Pointer Arithmetic with Char

```
char *str = “HELLO”;

str -> points to ‘H’
(str + 1) -> points to ‘E’
(str + 2) -> points to ‘L’

printf(“%s”, str+1); // Output : ELLO
```

```
printf("%s", str+2); // Output : LLO

printf("%s", *str); // Undefined Behaviour,
because %s expects a char pointer pointing to a
null-terminated string.

printf("%c", *str); // Output : H

printf("%c", *(str+1)); // Output : E
```

NOTE : Common Pitfalls

1.) Passing wrong type to %s

```
char* str = "HELLO";
printf("%s", *str); // Undefined Behaviour or
crash
```

2.) Trying to modify a string literal pointed by a character pointer

```
char *str = "HELLO";
str[1] = 'A'; // Error, string literal is
stored in read-only memory
```

3.)

```
char *ch;
    *ch = 'A'; // constants do not have address
```

NOTE : Pointer to Constant & Constant Pointer

(A) Pointer to Const Char

```
char str[] = "HELLO";
const char *ptr = str;
ptr[1] = 'A'; // ERROR
ptr = "HI"; // valid
```

We cannot modify the value, but the pointer can point to a different address.

(B) Const Pointer to Char

```
char str[] = “HELLO”;
char * const ptr = str;
ptr[1] = ‘a’; // valid
str = “HI”; // ERROR; cannot modify the
```

pointer(address can’t be changed). But the values can be modified.

(C) Const Pointer to Const Char

```
Char str[] = “HELLO”;
const char * const str = str;
str = “HI”; // INVALID
str[0] = ‘A’; // INVALID
-> Pointer and Data both can’t be changed
```

NOTE : Tricky Questions

1.)
```
char *p = "hello";
printf("%c", *p + 1);
```

Output : i

2.)
```
char arr[] = "abc";
char *p = arr;
p[1] = 'z';
printf("%s", arr);
```

Output : azc

3.) Is p[i] the same as i[p]? (IMPORTANT)

Answer : YES

p[i] = *(p + i)

i[p] = *(i + p)

4.) char *p = "hello";
printf("%c", *++p);

Output : e (Read Pointer Arithmetic Discussed in the class 😉)